“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS

PRÁCTICA SEMANA 09

Tecnología Web Backend, Arquitectura Cliente-Servidor, Servidores Web, PHP & JSP, Ciclo de Vida y Despliegue

ESTUDIANTE

Gago Uribe Edgar Robert

DOCENTE

Mg. Miguel Aguilar Coronación

ASIGNATURA

Desarrollo de Aplicaciones Web

HUANCAYO - PERÚ

2026-II

UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS

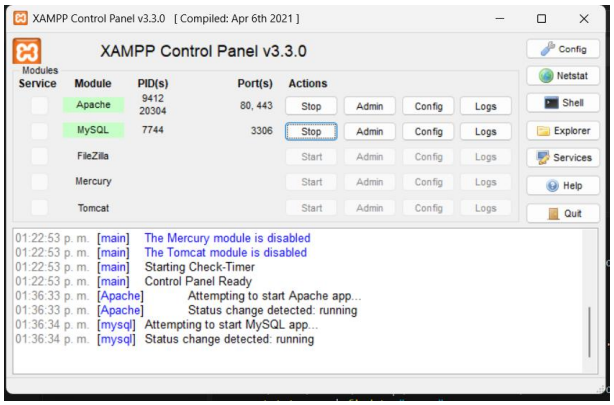
OBJETIVO DE LA PRÁCTICA:

1. Implementar y desplegar aplicaciones web del lado del servidor utilizando PHP y JSP, comprendiendo la arquitectura cliente-servidor, el ciclo de solicitud-respuesta HTTP, la configuración de entornos de ejecución (XAMPP/Laragon para PHP, Apache Tomcat para JSP), y validando el despliegue local con buenas prácticas de seguridad, manejo de sesiones y estructura básica MVC. Preparar la base para el desarrollo fullstack y consumo de APIs en semanas siguientes.

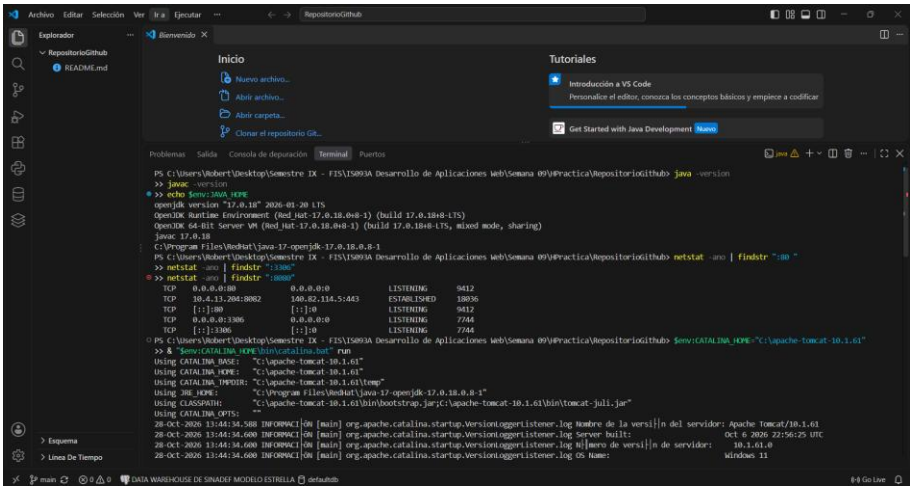
EJERCICIO PASO A PASO

1. Iniciar servidores: XAMPP (Apache/MySQL) y Tomcat (startup.sh/startup.bat). Verifica localhost y localhost:8080

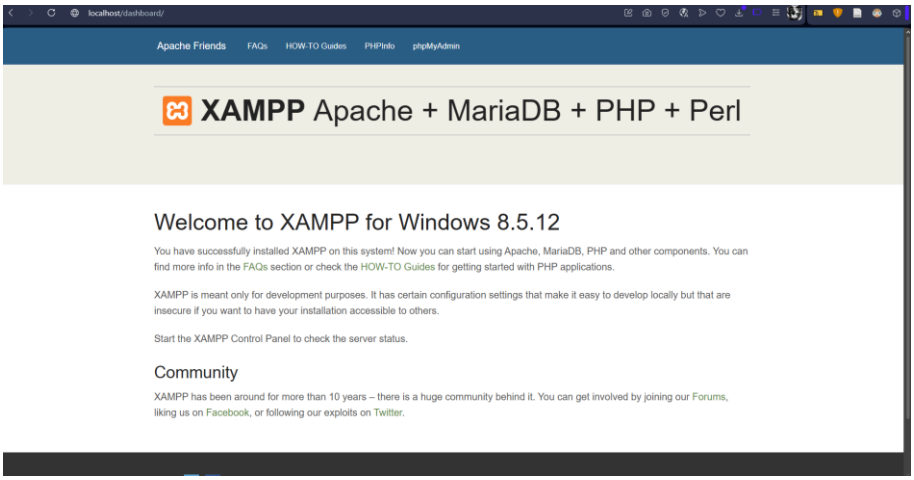
Iniciando XAMPP



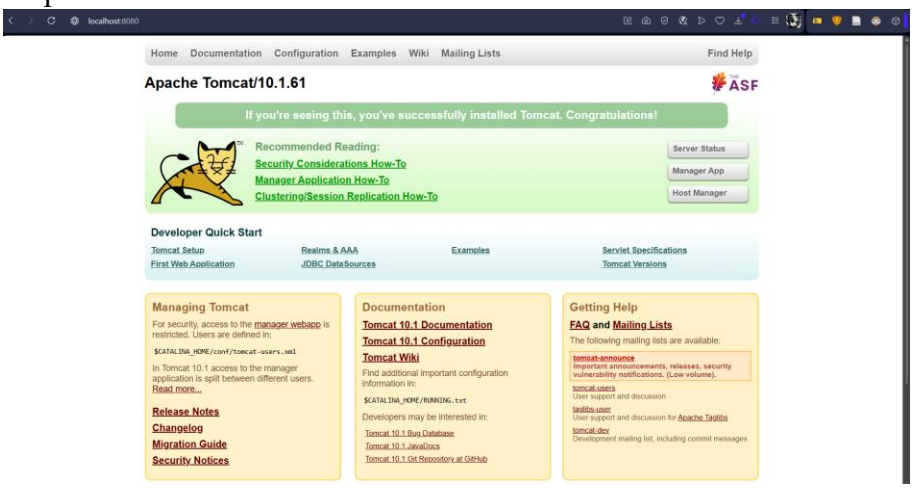
Iniciando Tomcat



Localhost con XAMPP

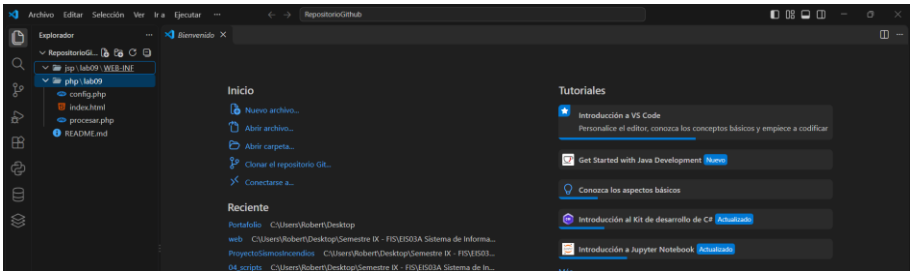


Localhost con Apache Tomcat

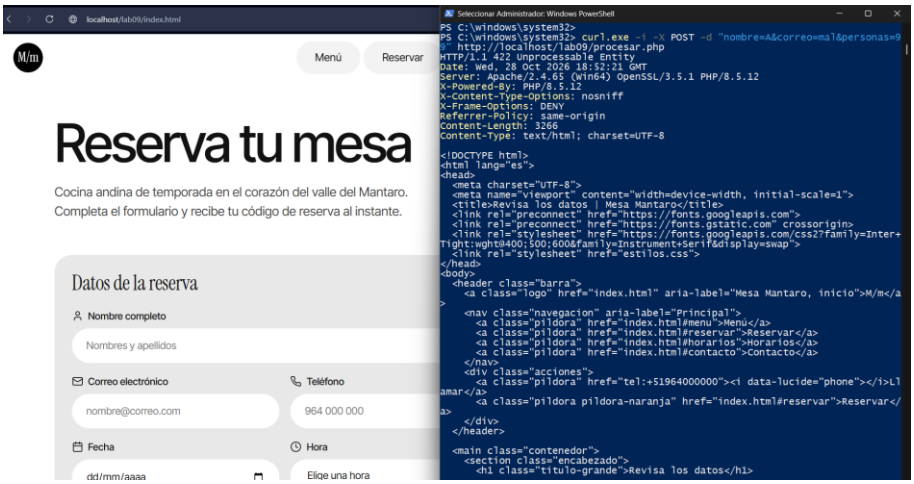


2. Crear estructura PHP: htdocs/lab09/, index.html, procesar.php, config.php. Implementar envío POST, validación básica y respuesta

Estructura PHP

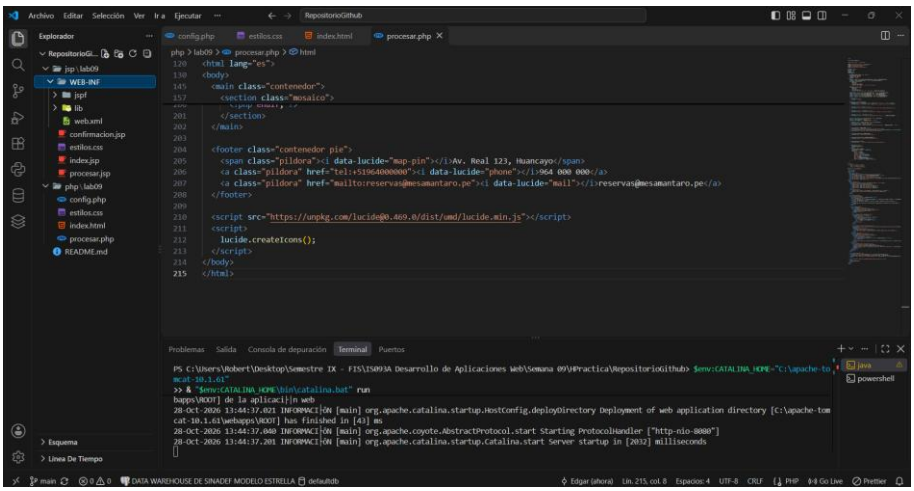


Envío POST



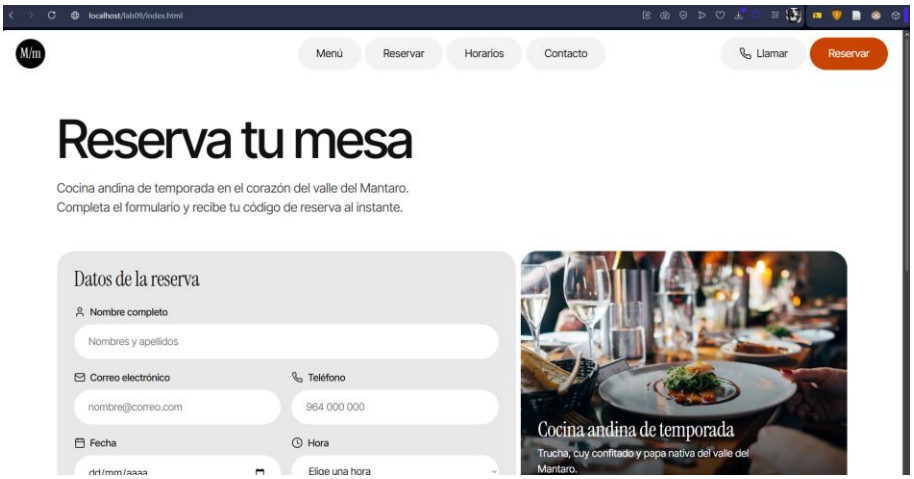
3. Crear estructura JSP: Tomcat/webapps/lab09/, index.jsp, procesar.jsp, WEBINF/web.xml (opcional). Implementar <%@ page %>, request.getParamete r(), session.setAttribute()

Estructura JSP

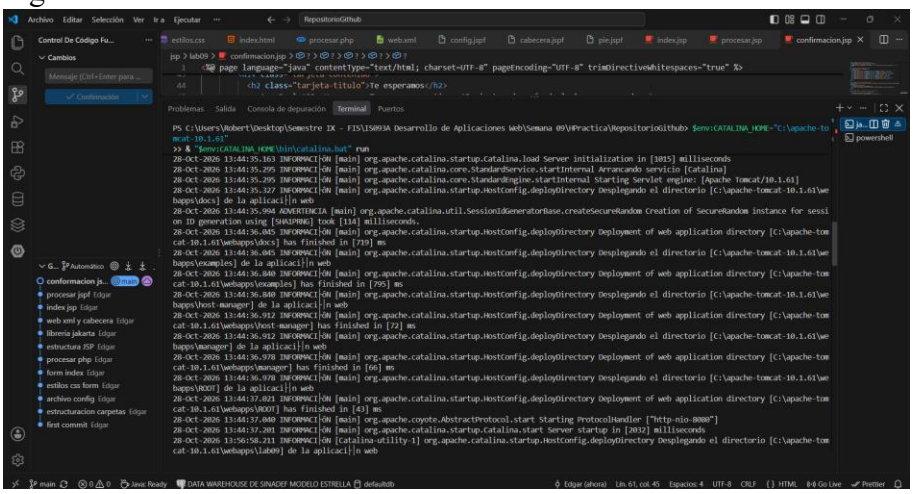


4. Desplegar y probar: Enviar formulario desde HTML/JSP, verificar respuesta, inspeccionar headers HTTP, revisar logs de acceso y error

Formulario HTML

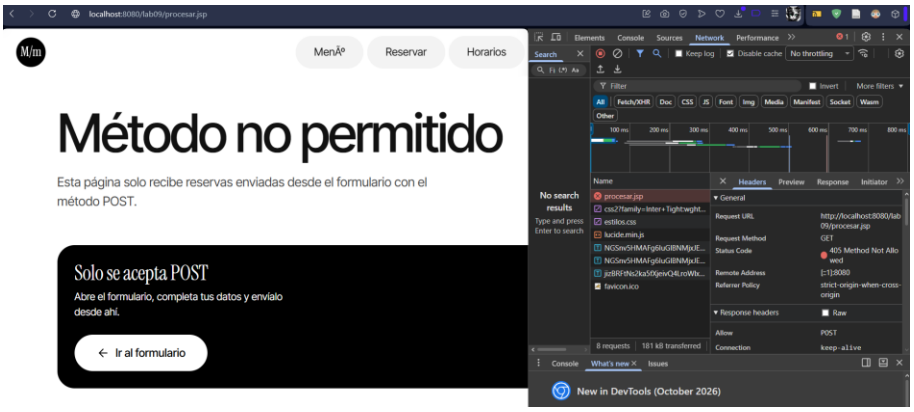


Verificando Logs de Servidor

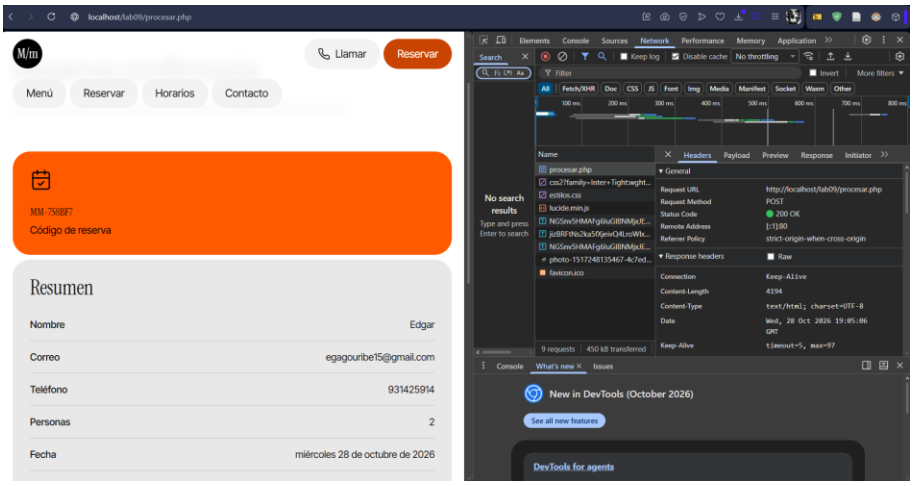


Inspeccionando Headers

Error 405



Envío Correcto de Formulario 200 OK



5. Comparar arquitecturas: Documentar diferencias de ciclo de vida, gestión de memoria, despliegue y escalabilidad entre PHP y JSP

Ciclo de vida. En PHP cada solicitud HTTP es independiente. Apache recibe la petición, el intérprete analiza el script (parse), lo ejecuta y devuelve el HTML, y al terminar todo el contexto se descarta, por eso procesar.php no recuerda nada entre envíos. En JSP, Tomcat traduce el archivo a un servlet (index_jsp.java), lo compila, crea una única instancia, atiende cada solicitud en el método service y solo lo destruye al detener o redesplegar la aplicación. Esa instancia persistente permite mantener estado, como el objeto session con la reserva en procesar.jsp.

Gestión de memoria. PHP sigue un modelo de nada compartido, La memoria se reserva al iniciar la solicitud y se libera por completo al terminar, limitada por memory_limit, lo que evita fugas acumuladas pero obliga a guardar el estado en $_SESSION, archivos o base de datos. JSP corre dentro de la JVM, donde el recolector de basura administra un heap compartido entre hilos y los objetos pueden vivir en los ámbitos request, session y application. Esto da más control y rendimiento, pero exige cuidar la concurrencia (variables declaradas con <%! %> son compartidas por todos los hilos) y evitar fugas de memoria.

Despliegue. Para PHP basta copiar los archivos a htdocs, sin compilación previa, y los cambios se ven al recargar la página, lo que simplifica el desarrollo. JSP requiere una estructura de aplicación web (WEB-INF, web.xml, librerías .jar como JSTL) dentro de webapps o empaquetada en un .war, además del JDK; Tomcat compila la página en la primera solicitud, así que esa primera respuesta es más lenta. A cambio, el despliegue es más formal y reproducible.

Escalabilidad. PHP escala horizontalmente con facilidad porque cada solicitud es independiente: se agregan servidores detrás de un balanceador, con sesiones externalizadas (por ejemplo en

Redis o base de datos) y OPcache para evitar recompilar scripts. JSP aprovecha hilos, pools de conexiones y cachés en memoria, lo que rinde bien con lógica de negocio pesada y muchos usuarios concurrentes. En un clúster necesita replicación de sesiones o afinidad de sesión, y la JVM consume más memoria base por instancia.

Link de GitHub:
https://github.com/E5gar/GuiaPracticaSemana09